

ELE113: FPGA - Mikrokontroller sammen med NIOS-II

Sondre Johnsen

May 18, 2026

Abstract

Til ELE-113 prosjektoppgaven valgte jeg selvvalgt prosjekt. Oppgaven var å bruke nios-2 til å styre en selvlaget CPU med data og program. Jeg ønsket også om mulig å bruke nios-2 til å vise hva CPUen har gjort for testing, samt for data-uthenting.

Først steg var å forstå mikrokontrolleres oppbygging og funksjon. Deretter det å programere VHDL filene, og simulere i UVVM. Avslutningsvis kunne jeg etter simuleringen også teste med NIOS-II på FPGAen og det virket uten problem. For å gjøre programmeringen lettere lagde jeg en egen kompiller.

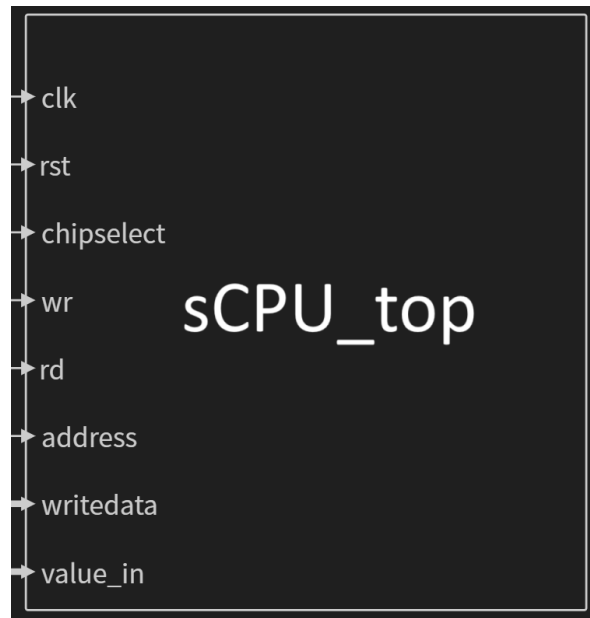


Figure 1: hvordan helheten ser ut

Contents

1	Introduksjon	3
1.1	Hva er bruksområdet	3
2	Planlegging og Design	3
2.1	Idé	3
2.2	Instruksjoner	4
3	Komponenter	4
3.1	ROM	5
3.2	Dekoder	5
3.3	Pointer	5
3.4	Registere	5
3.5	RAM / IO	5
3.6	ALU	6
3.7	Tegninger	6
4	VHDL kode	7
4.1	Rar / merkverdig kode	7
4.2	Full Adder	8
5	Koble opp til Nios-II	9
5.1	Ny tegning	9
5.2	Komponenter som ble lagt til eller som trengte endring	10
5.2.1	Data Strøm	10
5.2.2	Ny ROM	10
6	Programeringspråket	11
7	Compiler	12
8	Simulering	13
8.1	UVVM	13
8.1.1	Første program	13
8.1.2	Andre program	14
8.1.3	Fibonacci sekvens program	15
8.1.4	Count ones program	15
8.1.5	Siste resultat fra alle simuleringene	15
9	Programmering med NIOS-II	16
9.1	Første program	16
9.2	Andre program	16
9.3	Fibonacci program	17
9.4	Count Ones program	17
9.5	Videre testing	17
10	Datablad	18
11	Oppsummering	18
12	Filer	19
13	Referanse	20

1 Introduksjon

Utgangspunktet for hvordan jeg har valgt å løse denne oppgaven er en interesse for logikk og funksjon i oppbygging av en datamaskin. Tanken er at den beste lærdommen vil være i å bygge en egen mikrokontroller, som skal være programmerbar og kunne utføre enkle oppgaver.

En mikrokontroller manipulerer data ved for eksempel å addere, subtrahere, benytte bolsk logikk og betingete operasjoner. Altså en slags mini-datamaskin.

En moderne CPU er ekstremt stor og komplisert, og utenfor området av denne oppgaven. Målet her er å lage en enkle "CPU", som kan utføre seksten operasjoner.

1.1 Hva er bruksområdet

Eksempler på bruksområder for mikrokontrollere er i fjernkontroller, vaskemaskiner, ovner, ved generell datainnsamling og i industrien. De inngår også som en sleep kontroller til en annen enhet som trekker mye strøm. For eksempel i enheter som opererer på batterier, solcellepaneler eller hvis man vil lage en CPU med noen lav-effekts kjerner og i andre prosjekter som har begrenset strøm-budsjett. Man kan også benytte dem dersom man ønsker å skalere minnet opp og ned veldig enkelt. Dette ettersom det er minnet som tar mest plass.

Mikrokontrolleren som blir laget og beskrevet i denne oppgaven er veldig enkel og kan dermed bli masseprodusert ekstremt billig. Ettersom det er en veldig liten og enkel krets kan man bruke mange forskjellige størrelser på chip produksjonen. Altså trenger man ikke den nyeste 2 eller 3 nm prosessen. Men det er mulig å benytte denne teknologien, og da kan flere tusenvis av denne typen mikrokontroller bli laget på en enkelt silikon wafer. Dette er interessant fordi et problem med produksjonen av mikrobrikker, ligger i kravet til feilfrie silikonplater. Feil på en slik silikonplate kan gjøre at chipper må kastes. Dersom feil ikke oppdages kan viktige prosesser svikte. Dette kan løses med overflødighet, det vil si ved at man kan koble flere av disse mikrobrikkene sammen. I tilfelle en av dem gjør en feil så kan de andre kompensere. Et eksempel her er romfartøyet ([13](#))

2 Planlegging og Design

2.1 Idé

Jeg vil prøve å lage en Mikrokontroller i parrallell med NIOS-II der jeg kan bruke NIOS-II som et mellomlag til PCen min som en type "USB kontroller". Eller jeg kan bruke NIOS-II som vanlig, men å bruke flere selvlagde mikrokontrolleren. Slik at det blir slags "multi core" prosessor som kan kjøre noe i parallel for å gjøre en total prosess raskere. Tanken er å lage denne med en lavnivå logikk, altså bruke hovedsakelig logiske porter. Jeg skal prøve å lage den slik at den bruker en klokkepuls per linje isteden for det vanlige 3-step fetch, decode and execute. Dette selv om man kunne få bedre bruk av minnet ved å gjøre det slik. På denne måten blir det enklere, og hvis alt tar en klokkepuls så trenger jeg ikke noe tilfelle der jeg må stoppe tellingen i pointeren. Da blir det er mye lettere å regne ut hvor lang tid programmene tar, uansett hva man skriver i koden.

Først går jeg gjennom planleggingsfasen, så hvordan VHDL kode skal bli laget. Deretter skal komponentene kobles opp og settes sammen, før det simuleres og systemet testes med Testbench samt Signal-tap. Dersom tiden strekker til også gå gjennom UVVM.

2.2 Instruksjoner

Figure 2: Instruksjonssett.

		11	10	9	8	7	6	5	4	3	2	1	0						
	INS																		
0	NOP	0	0	0	0	0	0	N/A	N/A	N/A	N/A	N/A	N/A	NOP				GJØR INGENTING / VENTER EN KLOKKEPULS	
1	ADD	0	0	0	0	0	1	X	X	Y	Y	Z	Z	$X \leftarrow Y + Z$				REG(X) FÅR VERDI TIL REG(Y) + REG(Z)	
2	NEG	0	0	0	0	1	0	X	X	Y	Y	N/A	N/A	$X \leftarrow -Y$				REG(X) FÅR VERDI TIL NEGATIV REG(Y)	
3	MOVE	0	0	0	0	1	1	X	X	Y	Y	N/A	N/A	$X \leftarrow Y$				REG(X) FÅR VERDI TIL REG(Y)	
4	AND	0	0	0	1	0	0	X	X	Y	Y	Z	Z	$X \leftarrow Y \text{ and } Z$				REG(X) FÅR VERDI TIL REG(Y) AND REG(Z)	
5	OR	0	0	0	1	0	1	X	X	Y	Y	Z	Z	$X \leftarrow Y \text{ or } Z$				REG(X) FÅR VERDI TIL REG(Y) OR REG(Z)	
6	XOR	0	0	0	1	1	0	X	X	Y	Y	Z	Z	$X \leftarrow Y \text{ xor } Z$				REG(X) FÅR VERDI TIL REG(Y) XOR REG(Z)	
7	NOT	0	0	0	1	1	1	X	X	Y	Y	N/A	N/A	$X \leftarrow \text{not } Y$				REG(X) FÅR VERDI TIL NOT REG(Y)	
8	LOAD	0	0	1	0	0	0	X	X	AD	AD	AD	AD	$X \leftarrow \text{RAM(ADD)}$				REG(X) FÅR VERDI TIL RAM(AD)	
9	SAVE	0	0	1	0	0	1	X	X	AD	AD	AD	AD	$\text{RAM(AD)} \leftarrow X$				RAM(AD) FÅR VERDI TIL REG(X)	
10	JUMP	0	1	0	0	N/A	N/A	S	S	S	S	S	S	JUMP(ADD)				HOPPER I KODEN TIL LINJE "S"	
11	JUMPZ	0	1	0	1	X	X	S	S	S	S	S	S	JUMPZ(ADD):X				HVIS REG(X) = 0 HOPPER TIL LINJE "S"	
12	JUMPN	0	1	1	0	X	X	S	S	S	S	S	S	JUMPN(ADD):X				HVIS REG(X) < 0 HOPPER TIL LINJE "S"	
13	JUMPP	0	1	1	1	X	X	S	S	S	S	S	S	JUMPP(ADD):X				HVIS REG(X) > 0 HOPPER TIL LINJE "S"	
14	DEF	1	0	X	X	V	V	V	V	V	V	V	V	$X \leftarrow \text{VAL}$				REG(X) FÅR VERI "V"	
15	RST	1	1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	RST				SLETTER ALT I REGISTERENE og RAM	
instruksjon			X	Y	Z	X, Y og Z er vilkårlige register av A, B, C eller D								AD	AD er adresse til RAM				
S		S er "pointer" til hvilke linje i koden som blir brukt										V	V er 8-bit verdi med 2-kompliment						

Denne mikrokontrolleren skal bare kunne gjøre de enkleste operasjoner, dette for at det ikke skal bli en gigantisk kode. Figure 2 viser hva jeg har valgt av operasjoner som skal kunne gjøres. Instruksjon 10,11,12 og 13 er Jump instruksjoner, der ser man at den kan bare hoppe mellom 6-bits adresse, som fører til at den lengste koden i dette tilfelle er 64 linjer med maskin-kode. Jeg har valgt 12 bits lange instruksjoner / (kode), fordi de lengste "Dip-Switchen"ene jeg fant er 12-bits lange.

Med disse enkle operasjonene kan man gjøre masse andre ting som er mye mere komplekst. Et eksempel er multiplikasjon, som i praksis bare er å summere et tall et gitt antall ganger slik: $a * b = \sum_1^a b$. Deling blir litt mere komplisert, men det er bare subtrahering gjentatte ganger intil summen = 0: ser etter divisor $d = a / b : 0 = a - \sum_1^d b$. Merk at man må avrunder byten hvis det blir rest eller overflod.

```

1  a = 20
2  b = 4
3  m = 0
4  d = 0
5
6  summ = a
7  while(summ > 0): # m = a * b
8      summ -= 1
9      m += b
10 print(m)
11
12 summ = a
13 while(summ > 0): # d = a/b
14     summ -= b
15     d += 1
16 print(d)

```

Figure 3: Enkel kode for for multiplikasjon og divisjon med bruk av addisjon og subtraksjon.

Denne koden kan ikke brukes med negative tall, og hvis det er tall som blir større enn 7 bits så flyter tallet over.

3 Komponenter

For å forstå de påfølgende skjemaene som presenteres må man vite hva en Mikrokontroller er bygget opp av. Det vil si hvordan den leser kode, hvordan den legger og oppbevarer ting i minnet, hvordan den manipulerer minnet osv. Kort forklart er komponentene som inngår:

1. ROM 3.1 lagrer koden som skal kjøres (maskinkode).

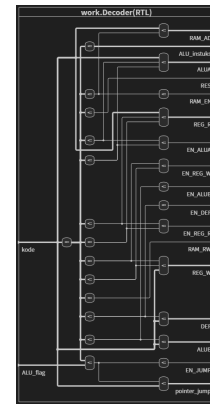
2. Dekoder 3.2 dekode instruksene til kontroller-signaler som kan bli brukt av andre enheter.
3. Pointer 3.3 holder kontroll på hvor i programmet man er (program counter).
4. Registerne 3.4 liten, rask lagringsplass inne i prosessoren som brukes til å holde data midlertidig under kjøring av programmet.
5. RAM 3.5, et arbeidsminne.
6. ALU 3.6 (Aritmetisk Logisk Enhet) kan manipulere og teste verdiene i registerene.

3.1 ROM

Skal ha 64 linjer/adresser med 12-bits ord som holder instruksjonene (Figure 2). Innganger, som ROM trenger, er bare en adresse. Utgangene er 12-bits ord. Merk at det ikke trengs synkronisering i denne komponenten.

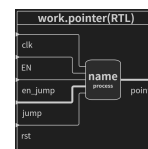
3.2 Dekoder

Leser rett fra ROM'en basert på en 12-bits "adresse", som den så dekode og som gir første linje av instruks (Gult i Fig 2). Måten den tester/sammenligner tallene på er ved hjelp av en stor "AND gate". Denne har innganger for der det skal være "1" og inverterte innganger for der det skal være "0". Disse OG-portene leser de første bitsene i instruksjonen (Gult i Fig 2). Når dekodere har funnet ut hvilke instruks som skal brukes, aktiverer den komponentene i følge resten av instruks. For eksempel for " $A = B + C$ "; først finner dekodere ut at det er en "add" instruks, så ser den på hvilke registre og hvor registerene (3.4) skal bli brukt. Før den aktiverer lesing på REG-A og sender REG-B og REG-C til ALU (3.6) inngang 1 og 2. Utgangene er altså å aktivere lesing og skriving på de 4 registerene til og fra RAM (3.5) og ALU og mellom hverandre.



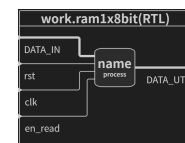
3.3 Pointer

Skal bare huske hvilke linje i koden man leser for øyeblikket. Normalt sett skal den øke i verdi for hver klokkepuls, men hvis det skjer en HOPP instruks så skal den lese inn den nye adressen og ikke addere en ekstra.



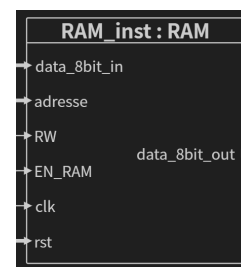
3.4 Registere

Er minne-blokker som blir aktivt jobbet på. For denne mikrokontrolleren har jeg valgt at det skal være 4 registre (REG-A, REG-B, REG-C og REG-D). De skal alle kunne kobles opp til en ”hoved-buss”, som alle andre registre, og som RAM kan lese og skrive fra og til. Alle registerene skal også kunne skrive til ALU (3.6) inngangene som er to uavhengige busserlinjer; ALU-Buss-A og ALU-Buss-B. Det er viktig at registerene er synkronisert til rising-edge.



3.5 RAM / IO

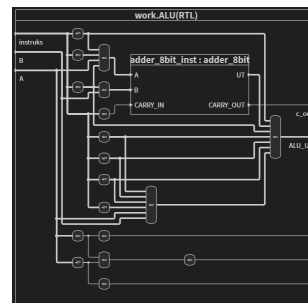
Skal bare ha 8-bits med 16 adresser (4-bits adresser, mørkeblå som vist i linje 8 og 9 i Fig 2) Inngangen skal være en adresse, read/write, og enable. Viktig med synkroisering, som blir aktivert av rising-edge. Denne heter RAM / IO fordi som med NIOS-II skal det være mulig å dele minnelokasjon med en annen enhet. Det



skal kunne være mulig å bruke minnelokasjoner både som innganger og som utganger. Om noen av minnelokasjonene brukes som innganger, er det viktig å huske å ikke skrive til de addressene.

3.6 ALU

Arithmetic Logic Unit (Aratmetisk logisk enhet). Gjør matematiske og bolske prossesser (se Fig 2, opperasjonene 1,2,4,5,6,7,11,12 og 13). Altså å addere (ADD), bytte fortegn (NEG), AND, OR, XOR, NOT og de betingete "Jump" operasjonene. Betinget hopp er if-setninger som hopper, styrt av ALU'en som tester om tallet inn er negativt, positivt, eller dersom tallet er null. ALU'en har to innganger der data fra registerene skal komme inn. Begge ingangene blir brukt dersom det er en "AND, OR, XOR" operasjon. Innstruksene NEG, NOT og test-JUMP bruker bare en av ingangene. JUMP-tesingene tester om tallet er lik null (JUMPZ), mindre enn null (negativ) (JUMPN) eller større enn null (JUMPP)



3.7 Tegninger

tegningene er her:

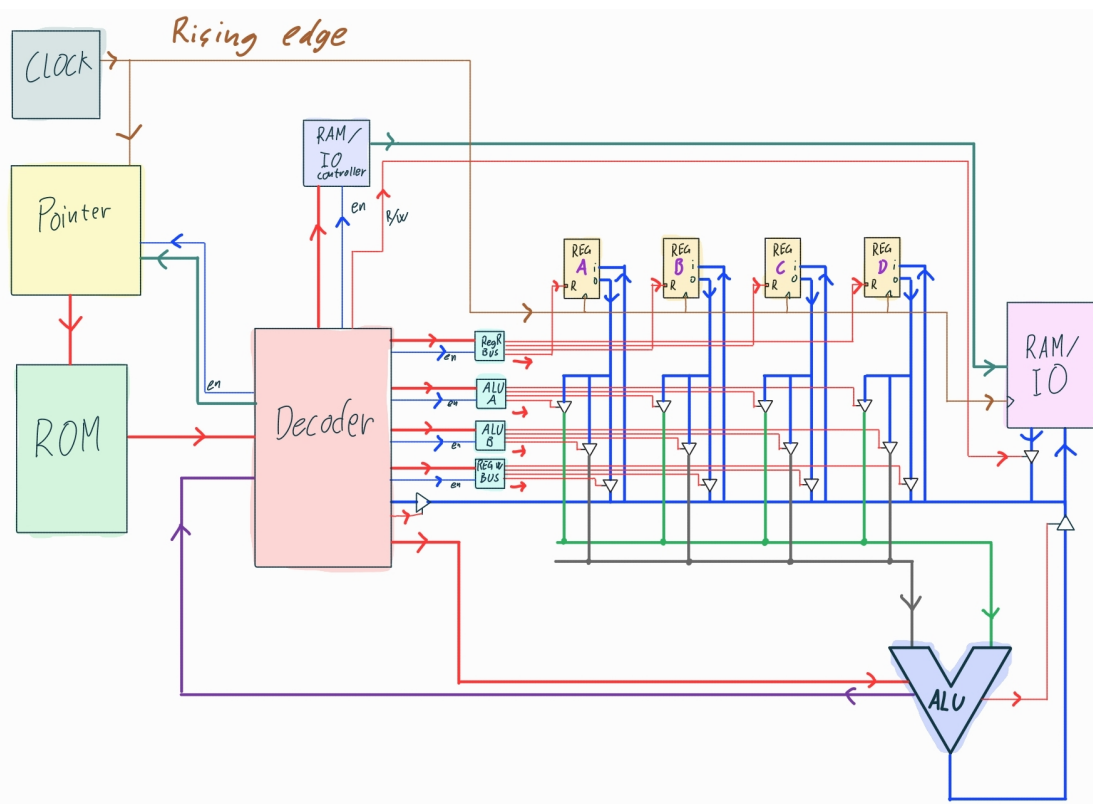


Figure 4: Plantegningen

Her vises planen, altså hva og hvordan alle komponentene skal kobles sammen i toppnivåfilen. Innganger og utganger fra hver komponent vises i tegningen. Noen av komponentene på tegningen, inneholder også andre mindre komponenter.

4 VHDL kode

4.1 Rar / merkverdig kode

Som planlagt ble koden hovedsaklig skrevet som bolsk logikk, som vanskeliggjør visualiseringen av hva som skjer. Nedenfor er et større eksempel på kode fra dekoderen jeg skal prøve å forklare gangen i:

```
instruks <= ((( (kode(11)) and (kode(10)) and (kode(9)) and (kode(8))) & -- res
((kode(11)) and (not kode(10))) & -- def
((( not kode(11)) and (kode(10)) and (kode(9)) and (kode(8))) & -- jump
((( not kode(11)) and (kode(10)) and (kode(9)) and (not kode(8))) & -- jumpn
((( not kode(11)) and (kode(10)) and (not kode(9)) and (kode(8))) & -- jumpz
((( not kode(11)) and (kode(10)) and (not kode(9)) and (not kode(8))) & -- jump
((((not kode(11)) and (not kode(10)) and (kode(9)) and (not kode(8)) and (not kode(7)) and (kode(6))) & -- save
((((not kode(11)) and (not kode(10)) and (kode(9)) and (not kode(8)) and (not kode(7)) and (not kode(6))) & -- load
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (kode(8)) and (kode(7)) and (kode(6))) & -- not
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (kode(8)) and (kode(7)) and (not kode(6))) & -- xor
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (kode(8)) and (not kode(7)) and (kode(6))) & -- or
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (kode(8)) and (not kode(7)) and (not kode(6))) & -- and
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (not kode(8)) and (kode(7)) and (kode(6))) & -- move
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (not kode(8)) and (kode(7)) and (not kode(6))) & -- neg
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (not kode(8)) and (not kode(7)) and (kode(6))) & -- add
((((not kode(11)) and (not kode(10)) and (not kode(9)) and (not kode(8)) and (not kode(7)) and (not kode(6))) ; -- noop
```

Figure 5: Viser hvordan instruksene blir dekodet

Denne binære-dekodeingen kan fremstå som kompleks eller uryddig, men er i realiteten bare mange OG-porter. Disse har som oppgave å sammenligne gitte instruksjoner og verdier av inkommende signal. Instruksene er i motsatt retning (av Fig 2) fordi element 0 kommer sist. En logisk representasjon av denne kretsen sees under:

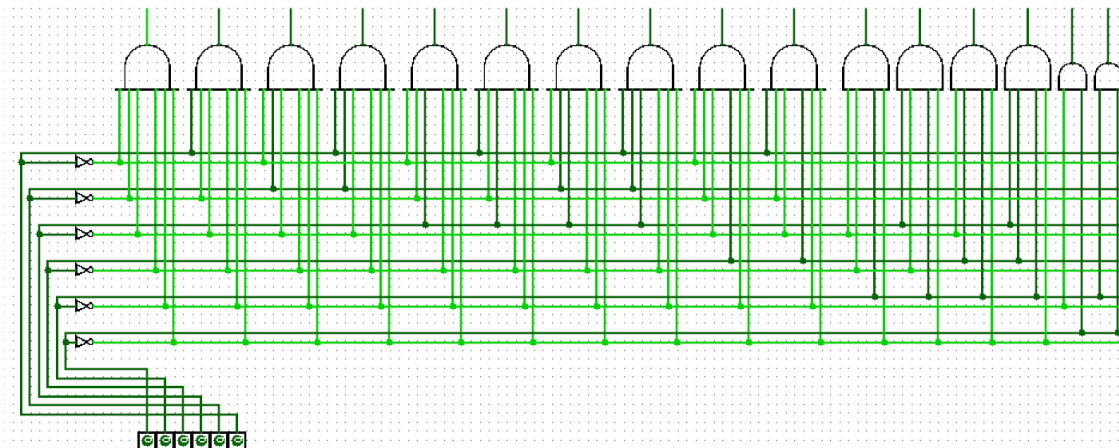


Figure 6

Denne delen av koden kunne blitt byttet med en "case" eller "ROM" for å gjøre det enklere å lese. Flere steder har lignende dekodere som for adressering av RAM (3.5) og registre (3.4) bare at de må ha en enable.

Flere steder ser man slike koder:

```
buss <= (DATA_UT0 and (decod_adr(0) & decod_adr(0) & decod_adr(0))) or
(DATA_UT1 and (decod_adr(1) & decod_adr(1) & decod_adr(1))) or
(DATA_UT2 and (decod_adr(2) & decod_adr(2) & decod_adr(2))) or
(DATA_UT3 and (decod_adr(3) & decod_adr(3) & decod_adr(3))) or
(DATA_UT4 and (decod_adr(4) & decod_adr(4) & decod_adr(4))) or
(DATA_UT5 and (decod_adr(5) & decod_adr(5) & decod_adr(5))) or
(DATA_UT6 and (decod_adr(6) & decod_adr(6) & decod_adr(6))) or
(DATA_UT7 and (decod_adr(7) & decod_adr(7) & decod_adr(7))) or
(DATA_UT8 and (decod_adr(8) & decod_adr(8) & decod_adr(8))) or
(DATA_UT9 and (decod_adr(9) & decod_adr(9) & decod_adr(9))) or
(DATA_UT10 and (decod_adr(10) & decod_adr(10) & decod_adr(10))) or
(DATA_UT11 and (decod_adr(11) & decod_adr(11) & decod_adr(11))) or
(DATA_UT12 and (decod_adr(12) & decod_adr(12) & decod_adr(12))) or
(DATA_UT13 and (decod_adr(13) & decod_adr(13) & decod_adr(13))) or
(DATA_UT14 and (decod_adr(14) & decod_adr(14) & decod_adr(14))) or
(DATA_UT15 and (decod_adr(15) & decod_adr(15) & decod_adr(15))) ;
```

Figure 7

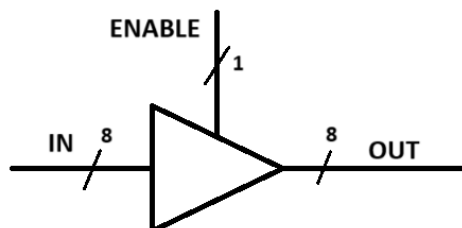


Figure 8

Figur 7(F 8) viser hvordan hver av linjene av koden (F 7) ovenifra ser ut som en tegning, urelatert til navn på inputs eller outputs

4.2 Full Adder

```
UTFRA(1) <= A nand E;
UTFRA(0) <= F nand G;
F <= D nand E;
G <= INTIL(0) nand E;
E <= INTIL(0) nand D;
D <= B nand C;
C <= INTIL(1) nand A;
B <= INTIL(2) nand A;
A <= INTIL(2) nand INTIL(1);
```

Figure 9

Full-adderen ble laget med bare IKKE-OG-porter, slik man ser til venstre (F 9). Dette fordi det er den billigste metoden å lage en full-adder-krets ifølge TTL-logikk.

Jeg har laget et kretskort som kan stables oppå hverandre, som vist til høyre (F 10).

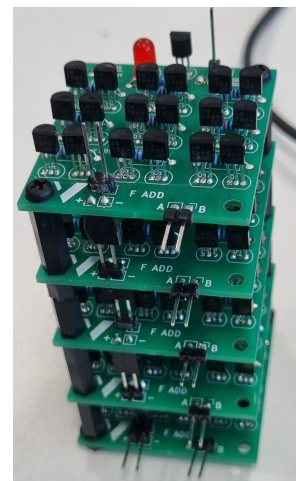


Figure 10

5 Koble opp til Nios-II

Så langt har jeg laget en kode som virker stand-alone uten NIOS-II. For å kunne koble den opp til NIOS-II som oppgaven krever, må de kunne kommunisere sammen ved hjelp av en SBI (simple bus interface). Det krever en del endringer, som beskrevet i de påfølgende avsnitt.

5.1 Ny tegning

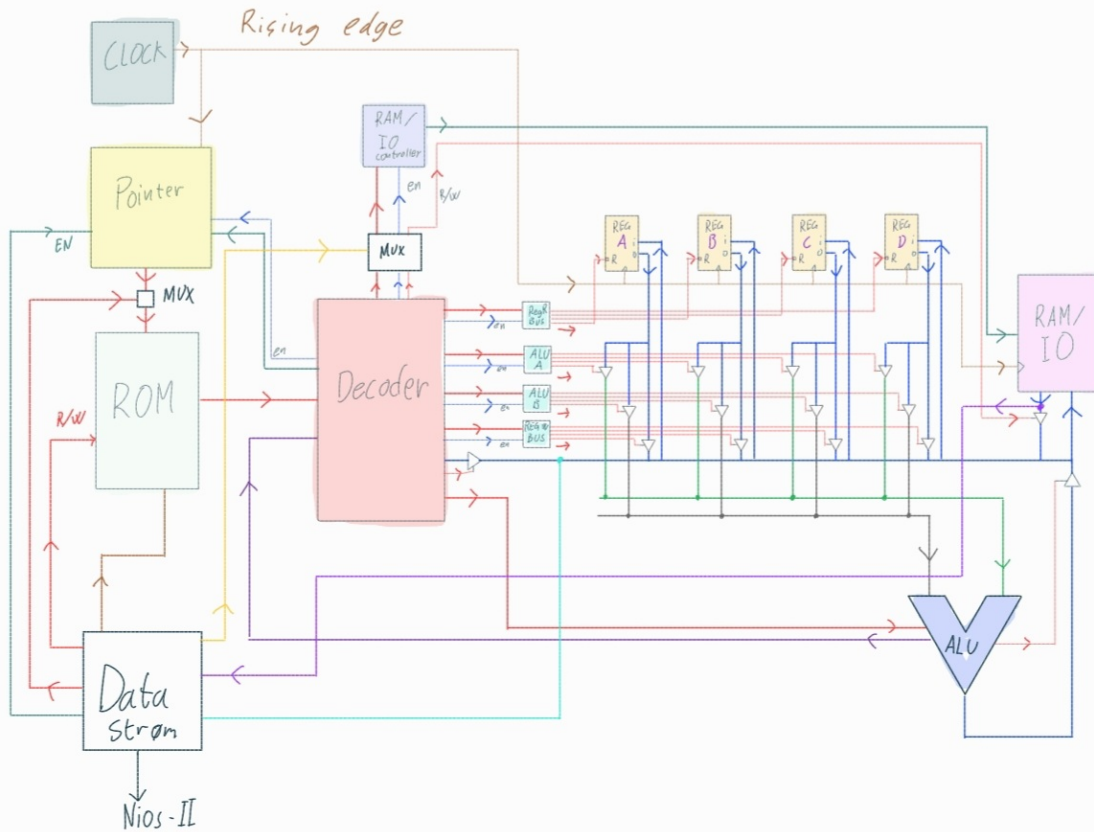


Figure 11: Nytt blokkdiagram for å få med sending og mottaking av data mellom NIOS-II og selvlaget CPU/Mikrokontroller.

Jeg må endre komponenter slik som det vises i den nye tegningen, jeg må gjøre slik at data skal kunne komme fra og til på SBI, jeg må endre ROM slik at jeg kan skrive et program inn i ROM, og til slutt kunne skrive til RAM og lese fra RAM. Dette beskrives i mer detaljer i de påfølgende avsnitt.

5.2 Komponenter som ble lagt til eller som trengte endring

5.2.1 Data Strøm

Måtte legge til komponenten som kan snakke med SBI. Denne komponenten heter Data-stroom. Den benytter bare en adresse på SBI. Den skal kunne skrive til ROM og RAM men bare lese fra RAM fordi det virker unødvendig å lese fra ROM og medføre mer arbeid.

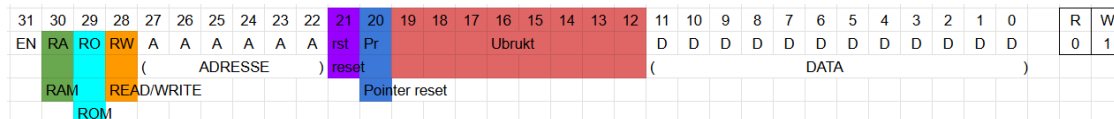


Figure 12: Hvordan man skriver til komponenten.

FIG 12 viser hvordan WRITEDATA skal se ut.

Når man skal lese fra RAM, må man sende en RAM-read request med adressen for at datastrøm skal stoppe mikrokontrolleren og hente data fra RAM. Etter en liten stund har Datastrøm funnet verdien og lagt den i "data" delen av registeret, deretter kan NIOS-II sender IORD, så leses read-data inn på NIOS-II. Man må AND verdien med 0xFF for å bare få de nederste 8 bitsene.

5.2.2 Ny ROM

Måtte endre ROM fra en typisk ROM som i ROM 7 SEG, dette fordi den må kunne lese inn maskinkode. Jeg har laget en komponent som gjør akkurat det som heter RAM 3.5 bare at den har 8-bits og kun 16 adresser. ROM skal ha 12-bits og 64 adresser. Så hvis jeg utvider 8-bit registerne i RAM til 12-bits, og så tar jeg 4 av disse 16 adresser 12-bits RAMene sammen. Samt setter ytterligere 2 bits til som adresse som velger mellom hvilke av de 16 adresse RAMene som blir brukt. Når alt er satt sammen får jeg en 64 adresse 12 bits RAM som jeg bruker som en ROM

Jeg bruker kun et register for å sende og motta data. For å snakke med komponenten må man slå den av / EN = '0', deretter kan man skrive til den. Man kan bare lese fra RAM, og derfor må man spørre om adressen så kan man lese verdien / skrive adresse for så å lese verdi i read-data. Data i posisjon 21 brukes til å restarte hele prosessoren, og pointer reset blir brukt til å bare restarte pointeren/program counteren.

6 Programeringspråket

Programeringsspråket er selvlaget. Jeg ville gjøre det lettere å skrive og lese selve programmet fordi binært maskinkode er ikke det letteste å skrive eller forstå.

MÅTER Å PROGRAMERE:

Opperasjon	Kode vising
NOP	NOP;
ADD	X <= Y + Z;
NEG	X <= Y + Z;
MOVE	X <= Y;
AND	X <= Y and Z;
OR	X <= Y or Z;
XOR	X <= Y xor Z;
NOT	X <= not Y;
LOAD	X <= RAM(ADD);
SAVE	RAM(ADD) <= X;
JUMP	JUMP(ADD);
JUMZ	JUMPZ(ADD):X;
JUMN	JUMPN(ADD):X;
JUMP	JUMPP(ADD):X;
DEF	X <= VAL;
RST	RST;

X,Y OG Z er vilkårlige registre (A ,B ,C og D) eksempel "X <= Y + Z;" -> "A <= B + C;" ADD er adresse, hvis ram blir brukt er det 0-15 adresser og for ROM er det 0-63 adresser VAL er verdi fra -128 til 127

7 Compiler

Jeg har laget, med litt help fra Chat-GPT [2] ([13](#)), en funksjonerende Compiler som leser inn mitt selvlagde program. Dette skriver jeg inn i en tekst (.txt) fil, så lager compileren det om til en maskinkode som NIOS-II kan bruke. Den første koden jeg testet ble for eksempel laget med compileren:

```
NOP;  
NOP;  
A <= 2;  
B <= 3;  
C <= 0;  
C <= A + B;  
RAM(1) <= C;  
NOP;
```

Figure 13

```
ROM_send(0b000000000000, 0x0000); // NOP;  
ROM_send(0b000000000000, 0x0001); // NOP;  
ROM_send(0b100000000010, 0x0002); // A <= 2;  
ROM_send(0b100100000011, 0x0003); // B <= 3;  
ROM_send(0b101000000000, 0x0004); // C <= 0;  
ROM_send(0b000001100001, 0x0005); // C <= A + B;  
ROM_send(0b001001100001, 0x0006); // RAM(1) <= C;  
ROM_send(0b000000000000, 0x0007); // NOP;
```

Figure 14

Til venste er koden jeg skrev inn i tekst dokument for å bli kompilert. Til høyre er maskinkoden som kom ut av compilingen. Alt man trenger å gjøre er å kopiere (ctrl + c) teksten fra "maskinkoden" og sette det inn i NIOS-II programmet. For å programere må man skrive nøyaktig det som instruksene sier. Hvis man skriver noe feil blir resultatet ? tegn, der man skrev noe feil. Så man kan bruke det for å finne hvor feilen er.

Koder er vedlagt. Den kommer med kode.txt som er tekstdokumentet man skriver inn i, maskinkode.txt er maskinkoden som kommer ut, og et python-program som man må kjøre for at koden skal bli kompilert. Viktig å huske at disse filene må være i samme overfil.

8 Simulering

8.1 UVVM

Jeg har satt opp noen enkle program for å teste om de ulike komponentene virker som forventet. I check ser man at de tre nederste hex tallene ikke blir testet, fordi i VHDL programmet så svarer den med den verdien som ikke skal inn. Altså kun teste bitsene når man leser fra mikrokontrolleren.

8.1.1 Første program

Dette programmet er mer som en test av ALU 3.6, SBI 5.2.1, Register 3.4, ROM 5.2.2, RAM 3.5, pointeren 3.3 og dekoderen 3.2. Det er et enkelt program som alle CPU'er burde kunne., man kan se det til høyre. Jeg skal bare sende inn dette programminnet, deretter skal jeg se om den gir ut riktig resultat som er $2 + 3$.

```
NOP;
NOP;
A <= 2;
B <= 3;
C <= 0;
C <= A + B;
RAM(1) <= C;
NOP;
```

```
write(0, x"30000000", "skriv adresse 0, 0x00");      -- sender inn NOP,      ADR 0x00
wait for 10*T;
check(0, x"30000000", ERROR, "sjekk NOP, adr 0x00");
wait for 100 *T;

write(0, x"30400000", "skriv adresse 0, 0x01");      -- sender inn NOP,      ADR 0x01
wait for 10*T;
check(0, x"30400000", ERROR, "sjekk NOP, adr 0x01");
wait for 100 *T;

write(0, x"30800002", "skriver A <= 2, adr 0x02");    -- sender inn A <= 2,    ADR 0x02
wait for 10*T;
check(0, x"30800000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"30C00003", "skriv B <= 3, adr 0x03");      -- sender inn B <= 3,    ADR 0x03
wait for 10*T;
check(0, x"30C00000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"31000000", "skriv C <= 0, adr 0x04");      -- sender inn C <= 0,    ADR 0x04
wait for 10*T;
check(0, x"31000000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"31400005", "skriv C <= A + B, adr 0x05");  -- sender inn C <= A + B, ADR 0x05
wait for 10*T;
check(0, x"31400000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"31800261", "skriv innhoid i C skal til ram adr 1, adr 0x06");  -- sender inn RAM(1) <= C, ADR 0x06
wait for 10*T;
check(0, x"31800000", ERROR, "sjekk adresse 0");
wait for 100 *T;

-- Starter prosessoren
write(0, x"00000000", "sender start");                -- sender inn start komando,
wait for 10*T;
check(0, x"00000000", ERROR, "sjekk start");
wait for 100 *T;

write(0, x"00000000", "sender stopp");                -- sender inn stopp komando,
wait for 100*T;
write(0, x"40400000", "sender request RAM adresse 1"); -- sender inn stopp komando,
wait for 10*T;
-- Leser resultat
check(0, x"40400005", ERROR, "sjekker resultatet, skal vare lik 5"); -- sjekker om det er kommet tilbake 2+3 da skal verdien vere 5
wait for 10*T;
```

Figure 15

```
# UVVM: ID LOG_HEX      420.0 ns TB seq.      Sjekke første program:
# UVVM: -----
# UVVM: ID_BPM          445.0 ns TB seq.      sbl_check(A[is"0", x"00000000"]=> OK, read data = x"00000000", "sjekk adresse 0"
# UVVM: ID_BPM          465.0 ns TB seq.      sbl_write(A[is"0", x"30000000"] completed. "skriv adresse 0, 0x00"
# UVVM: ID_BPM          695.0 ns TB seq.      sbl_check(A[is"0", x"30000000"]=> OK, read data = x"30000000", "sjekk NOP, adr 0x00"
# UVVM: ID_BPM          2705.0 ns TB seq.      sbl_write(A[is"0", x"30400000"] completed. "skriv adresse 0, 0x01"
# UVVM: ID_BPM          2925.0 ns TB seq.      sbl_check(A[is"0", x"30400000"]=> OK, read data = x"30400000", "sjekk NOP, adr 0x01"
# UVVM: ID_BPM          4945.0 ns TB seq.      sbl_write(A[is"0", x"30800002"] completed. "skriver A <= 2, adr 0x02"
# UVVM: ID_BPM          5165.0 ns TB seq.      sbl_check(A[is"0", x"30800000"]=> OK, read data = x"30800000", "sjekk"
# UVVM: ID_BPM          7185.0 ns TB seq.      sbl_write(A[is"0", x"30C00003"] completed. "skriv B <= 3, adr 0x03"
# UVVM: ID_BPM          7405.0 ns TB seq.      sbl_check(A[is"0", x"30C00000"]=> OK, read data = x"30C00000", "sjekk"
# UVVM: ID_BPM          9425.0 ns TB seq.      sbl_write(A[is"0", x"31000000"] completed. "skriv C <= 0, adr 0x04"
# UVVM: ID_BPM          9645.0 ns TB seq.      sbl_check(A[is"0", x"31000000"]=> OK, read data = x"31000000", "sjekk"
# UVVM: ID_BPM          11665.0 ns TB seq.      sbl_write(A[is"0", x"31400005"] completed. "skriv C <= A + B, adr 0x05"
# UVVM: ID_BPM          11885.0 ns TB seq.      sbl_check(A[is"0", x"31400000"]=> OK, read data = x"31400000", "sjekk"
# UVVM: ID_BPM          13905.0 ns TB seq.      sbl_write(A[is"0", x"31800261"] completed. "skriv innhoid i C skal til ram adr 1, adr 0x06"
# UVVM: ID_BPM          14125.0 ns TB seq.      sbl_check(A[is"0", x"31800000"]=> OK, read data = x"31800000", "sjekk adresse 0"
# UVVM: ID_BPM          16145.0 ns TB seq.      sbl_write(A[is"0", x"00000000"] completed. "sender start"
# UVVM: ID_BPM          16365.0 ns TB seq.      sbl_check(A[is"0", x"00000000"]=> OK, read data = x"00000000", "sjekk start"
# UVVM: ID_BPM          18385.0 ns TB seq.      sbl_write(A[is"0", x"00000000"] completed. "sender stopp"
# UVVM: ID_BPM          20405.0 ns TB seq.      sbl_write(A[is"0", x"40400000"] completed. "sender request RAM adresse 1"
# UVVM: ID_BPM          20625.0 ns TB seq.      sbl_check(A[is"0", x"40400005"]=> OK, read data = x"40400005", "sjekker resultatet, skal være lik 5"
# UVVM: ID GEN PULSE      20845.0 ns TB seq.      Pulsed to 1 for 20 ns. "Reset"
```

Figure 16

→ Alt virket.

8.1.2 Andre program

I det andre programmet er det samme som jeg jeg simulerte der, testet jeg blant annet overskrivningen av RAM data, og om det virker å hoppe til seg selv i programmet aka en "halt" i programmet. Den får inn 100 i RAM adresse 1, og så leser den det ut og plusser på 3 så legger den det tilbake til i RAM plass 1 og stopper Program telleren (program counter).

```
NOP;
A <= RAM(1);
B <= 3;
C <= A + B;
RAM(1) <= C;
JUMP(5);
```

```
-- skriver til RAM
write(0, x"50400064", "skriver inn 100 i ram adresse 0x1"); -- setter inn 100 i ram adresse 1
wait for 10*T;
check(0, x"50400000", ERROR, "sjekk NOOP, adr 0x00");
wait for 100 *T;

-- skriver til ROM:
write(0, x"30000000", "NOP, 0x00"); -- sender inn NOP, ADR 0x00
wait for 10*T;
check(0, x"30000000", ERROR, "sjekk NOOP, adr 0x00");
wait for 100 *T;

write(0, x"30400201", "A <= RAM(0), 0x01"); -- A får verdi til RAM adresse 1, ADR 0x01
wait for 10*T;
check(0, x"30400000", ERROR, "sjekk NOOP, adr 0x01");
wait for 100 *T;

write(0, x"30800003", "B <= 3, adr 0x02"); -- B = 3, ADR 0x02
wait for 10*T;
check(0, x"30800000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"30C00061", "C <= A + B, adr 0x03"); -- C = A + B, ADR 0x03
wait for 10*T;
check(0, x"30C00000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"31000261", "RAM(1) <= C, adr 0x04"); -- sender inn C <= 0, ADR 0x04
wait for 10*T;
check(0, x"31000000", ERROR, "sjekk");
wait for 100 *T;

write(0, x"31400405", "JUMP(5), adr 0x05"); -- halter programmet fordi den hopper til seg selv
wait for 10*T; -- ADR 0x05
check(0, x"31400000", ERROR, "sjekk");
wait for 100 *T;

-- Starter prosessoren
write(0, x"80000000", "sender start"); -- sender inn start komando,
wait for 10*T;
check(0, x"80000000", ERROR, "sjekk start");
wait for 100 *T;

write(0, x"00000000", "sender stopp"); -- sender inn stopp komando,
wait for 100*T;
write(0, x"40400000", "sender request RAM adresse 1"); -- sender inn stopp komando,
wait for 10*T;
-- leser resultat -- sjekker om det er kommet tilbake 100+3 da skal verdien være 103
check(0, x"40400067", ERROR, "sjekker resultatet, skal være lik 103");
wait for 10*T;
```

Figure 17

#	UVVM: ID_LOG_HDR	21045.0 ns	TB seq.	Tester andre program:
#	UVVM: ID_BFM	21265.0 ns	TB seq.	abi_write(A:0, x"50400064") completed. 'skriver inn 100 i ram adresse 0x1'
#	UVVM: ID_BFM	21495.0 ns	TB seq.	abi_check(A:0, x"50400000")=> OK, read data = x"50400000". 'sjekk NOOP, adr 0x00'
#	UVVM: ID_BFM	23505.0 ns	TB seq.	abi_write(A:0, x"30000000") completed. 'NOP, 0x00'
#	UVVM: ID_BFM	23725.0 ns	TB seq.	abi_check(A:0, x"30000000")=> OK, read data = x"30000000". 'sjekk NOOP, adr 0x00'
#	UVVM: ID_BFM	25745.0 ns	TB seq.	abi_write(A:0, x"30400201") completed. 'A <= RAM(0), 0x01'
#	UVVM: ID_BFM	25965.0 ns	TB seq.	abi_check(A:0, x"30400000")=> OK, read data = x"30400000". 'sjekk NOOP, adr 0x01'
#	UVVM: ID_BFM	27985.0 ns	TB seq.	abi_write(A:0, x"30800003") completed. 'B <= 3, adr 0x02'
#	UVVM: ID_BFM	28205.0 ns	TB seq.	abi_check(A:0, x"30800000")=> OK, read data = x"30800000". 'sjekk'
#	UVVM: ID_BFM	30225.0 ns	TB seq.	abi_write(A:0, x"30C00061") completed. 'C <= A + B, adr 0x03'
#	UVVM: ID_BFM	30445.0 ns	TB seq.	abi_check(A:0, x"30C00000")=> OK, read data = x"30C00000". 'sjekk'
#	UVVM: ID_BFM	32465.0 ns	TB seq.	abi_write(A:0, x"31000261") completed. 'RAM(1) <= C, adr 0x04'
#	UVVM: ID_BFM	32685.0 ns	TB seq.	abi_check(A:0, x"31000000")=> OK, read data = x"31000000". 'sjekk'
#	UVVM: ID_BFM	34705.0 ns	TB seq.	abi_write(A:0, x"31400405") completed. 'JUMP(5), adr 0x05'
#	UVVM: ID_BFM	34925.0 ns	TB seq.	abi_check(A:0, x"31400000")=> OK, read data = x"31400000". 'sjekk'
#	UVVM: ID_BFM	36945.0 ns	TB seq.	abi_write(A:0, x"80000000") completed. 'sender start'
#	UVVM: ID_BFM	37165.0 ns	TB seq.	abi_check(A:0, x"80000000")=> OK, read data = x"80000000". 'sjekk start'
#	UVVM: ID_BFM	39185.0 ns	TB seq.	abi_write(A:0, x"00000000") completed. 'sender stopp'
#	UVVM: ID_BFM	41205.0 ns	TB seq.	abi_write(A:0, x"40400000") completed. 'sender request RAM adresse 1'
#	UVVM: ID_BFM	41425.0 ns	TB seq.	abi_check(A:0, x"40400067")=> OK, read data = x"40400067". 'sjekker resultatet, skal være lik 103'
#	UVVM:			

Figure 18

→ Alt virket fint.

8.1.3 Fibonacci sekvens program

Jeg ville som i oppgaveteksten jeg gav til meg selv prøve å lage et program som kan finne alle fibonacci tallene helt opp til 8-bits 2's komplement. For å få denne til å virke må man førs sette inn 0 på RAM adresse 0x0 og 1 på RAM adresse 0x1.

Resultater var bra. Det går ann å se i UVVM koden.

```
A <= RAM(0);
B <= RAM(1);
C <= A + B;
JUMPN(8):C;
B <= A;
A <= C;
RAM(0) <= A;
RAM(1) <= B;
JUMP(8);
```

8.1.4 Count ones program

Jeg prøvde også å teste det å telle bits i en 8-bit byte. For å gjøre det kan man teste om tallet er negativt fordi da er den øverste bitt høy, så må man skifte alle bittene oppover med en. For å skifte bittene kan man gjøre en enkel $A = A + A$ som er det samme som $A = 2 * A$, og det er det samme som å shifte alle bittene oppover med en. Resultater finnes i UVVM fil.

```
D <= -8;
C <= 1;
B <= 0;
A <= RAM(0);
JUMPZ(10):D;
JUMPN(7):A;
JUMP(8);
B <= B + C;
D <= D + C;
A <= A + A;
RAM(2) <= B;
JUMP(4);
```

8.1.5 Siste resultat fra alle simuleringene

```

# UVVM: =====
# UVVM: *** FINAL SUMMARY OF ALL ALERTS ***
# UVVM: =====
# UVVM:
# UVVM:                REGARDED    EXPECTED    IGNORED    Comment?
# UVVM:    NOTE          :         0         0         0      ok
# UVVM:    TB_NOTE       :         0         0         0      ok
# UVVM:    WARNING       :         0         0         0      ok
# UVVM:    TB_WARNING    :         0         0         0      ok
# UVVM:    MANUAL_CHECK   :         0         0         0      ok
# UVVM:    ERROR         :         0         0         0      ok
# UVVM:    TB_ERROR       :         0         0         0      ok
# UVVM:    FAILURE       :         0         0         0      ok
# UVVM:    TB_FAILURE    :         0         0         0      ok
# UVVM:
# UVVM: =====
# UVVM: >> Simulation SUCCESS: No mismatch between counted and expected serious alerts
# UVVM: =====

```

Figure 19

Dette viser at alle simuleringene ovenfor virket.
Mer informasjon finnes i UVVM filen.

9 Programmering med NIOS-II

Jeg programerte alt i prosjektfilen: Prosjekt-ele113-sondre. ([12](#))
Her viser jeg en del test program for å vise at hele prosjektet virker.

9.1 Første program

Det første programmet er det samme programmet som jeg simulerte med UVVM. Programmet ble laget med kompileringen ([7](#)) som jeg har laget.

```
21 ROM_send(0b000000000000, 0x0000); // NOP;
22 ROM_send(0b000000000000, 0x0001); // NOP;
23 ROM_send(0b1000000000010, 0x0002); // A <= 2;
24 ROM_send(0b1001000000011, 0x0003); // B <= 3;
25 ROM_send(0b1010000000000, 0x0004); // C <= 0;
26 ROM_send(0b000001100001, 0x0005); // C <= A + B;
27 ROM_send(0b001001100001, 0x0006); // RAM(1) <= C;
28 ROM_send(0b000000000000, 0x0007); // NOP;
29 start();
30
31 stopp();
32 int ut = RAM_motta(0x00000001);
33 printf("skal gi ut 5 = %i\n",ut);
```

Figure 20: Selve koden som er lastet opp.

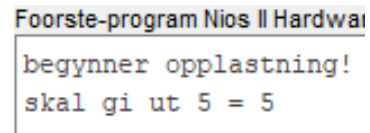


Figure 21: Resultatet av første test.

Her ser man den enkleste testen som viser at den kan ihvertfall skrive inn til ROM og lese ut fra RAM

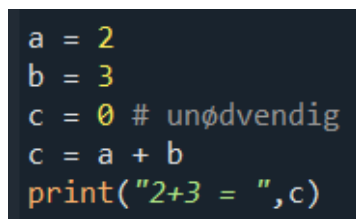


Figure 22: Ekvivalent python program



Figure 23: Resultat i python

9.2 Andre program

```
20 RAM_send(100,1);
21
22 ROM_send(0b000000000000, 0x0000); // NOP;
23 ROM_send(0b001000000001, 0x0001); // A <= RAM(1);
24 ROM_send(0b1001000000011, 0x0002); // B <= 3;
25 ROM_send(0b000001100001, 0x0003); // C <= A + B;
26 ROM_send(0b001001100001, 0x0004); // RAM(1) <= C;
27 ROM_send(0b010000000101, 0x0005); // JUMP(5);
28
29 start();
30
31 stopp();
32
33 int ut = RAM_motta(0x00000001);
34 printf("skal gi ut 103 : %i\n",ut);
```

Figure 24: Program inn

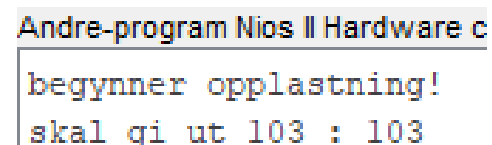


Figure 25: Svaret.

Samme som i UVVM. Her har jeg lagt inn en verdi 100 i RAM adresse 0, deretter leser den det inn i register A. B blir 3, så plusser den A+B og legger svaret i RAM adresse 1. NIOS-II leser fra RAM adresse 1 og får ut riktig svar altså 103 fordi $100 + 3 = 103$.

Nå har jeg vist at man kan skrive inn i ROMen, skrive og lese fra / til RAM og testet summering i ALU-en.

9.3 Fibonacci program

```
20 RAM_send(0,0);
21 RAM_send(1,1);
22
23 ROM_send(0b001000000000, 0x0000); // A <= RAM(0);
24 ROM_send(0b001000010001, 0x0001); // B <= RAM(1);
25 ROM_send(0b000001100001, 0x0002); // C <= A + B;
26 ROM_send(0b011010001000, 0x0003); // JUMPN(8):C;
27 ROM_send(0b000011010000, 0x0004); // B <= A;
28 ROM_send(0b000011001000, 0x0005); // A <= C;
29 ROM_send(0b001001000000, 0x0006); // RAM(0) <= A;
30 ROM_send(0b001001010001, 0x0007); // RAM(1) <= B;
31 ROM_send(0b010000010000, 0x0008); // JUMP(8);
32
33 for (int i = 0; i < 15; i++) {
34     start();
35     stopp();
36     int ut = RAM_motta(0x00000001);
37     printf("%d av fibonacci gir ut %i\n", i, ut);
38     point_restart();
39 }
40
```

Figure 26: Program inn

Fibonacci-program Nios II Hardware configura

```
begynner opplasting!
0 av fibonacci gir ut 0
1 av fibonacci gir ut 1
2 av fibonacci gir ut 1
3 av fibonacci gir ut 2
4 av fibonacci gir ut 3
5 av fibonacci gir ut 5
6 av fibonacci gir ut 8
7 av fibonacci gir ut 13
8 av fibonacci gir ut 21
9 av fibonacci gir ut 34
10 av fibonacci gir ut 55
11 av fibonacci gir ut 55
12 av fibonacci gir ut 55
```

Figure 27: Svaret.

9.4 Count Ones program

```
23 ROM_send(0b101111111000, 0x0000); // D <= -8;
24 ROM_send(0b101000000001, 0x0001); // C <= 1;
25 ROM_send(0b100100000000, 0x0002); // B <= 0;
26 ROM_send(0b001000000000, 0x0003); // A <= RAM(0);
27 ROM_send(0b010111001010, 0x0004); // JUMP2(10):D;
28 ROM_send(0b011000000111, 0x0005); // JUMPN(7):A;
29 ROM_send(0b010000001000, 0x0006); // JUMP(8);
30 ROM_send(0b000001010110, 0x0007); // B <= B + C;
31 ROM_send(0b000001111110, 0x0008); // D <= D + C;
32 ROM_send(0b000001000000, 0x0009); // A <= A + A;
33 ROM_send(0b001001010010, 0x000a); // RAM(2) <= B;
34 ROM_send(0b010000000100, 0x000b); // JUMP(4);
35
36 RAM_send(0x62,0);
37 start();
38 stopp();
39
40 int ut = RAM_motta(0x00000002);
41 printf("skal gi ut 0x62 har 3 hooye bits og resulatet er : %i\n",ut);
42
```

Figure 28: Program inn

Problems Tasks Console Nio

Count_ones-program Nios II Hardware configuration - cable

```
tall 234 har 5 anntal hooye bits
tall 235 har 6 anntal hooye bits
tall 236 har 5 anntal hooye bits
tall 237 har 6 anntal hooye bits
tall 238 har 6 anntal hooye bits
tall 239 har 7 anntal hooye bits
tall 240 har 4 anntal hooye bits
tall 241 har 5 anntal hooye bits
tall 242 har 5 anntal hooye bits
tall 243 har 6 anntal hooye bits
tall 244 har 5 anntal hooye bits
tall 245 har 6 anntal hooye bits
tall 246 har 6 anntal hooye bits
tall 247 har 7 anntal hooye bits
tall 248 har 5 anntal hooye bits
tall 249 har 6 anntal hooye bits
tall 250 har 6 anntal hooye bits
tall 251 har 7 anntal hooye bits
tall 252 har 6 anntal hooye bits
tall 253 har 7 anntal hooye bits
tall 254 har 7 anntal hooye bits
```

Figure 29: Svaret.

```
printf("alle tall fra 0-255");
for (int i = 0; i < 255; i++) {
    point_restart();
    RAM_send(i,0);
    start();
    stopp();
    ut = RAM_motta(0x00000002);
    printf("tall %d har %i anntal hooye bits\n",i,ut);
}
```

Figure 30: resten av programmet

Dette programmet var spesielt gøy å klare, fordi i oblig-2 fikk vi samme oppgave bare at vi skulle bruke NIOS-II

9.5 Videre testing

Jeg har testet alle mulige koder som gjør forskjellige ting og det viser seg at alle funksjonene virker knirkefritt. Alle disse programmene er tatt med i innleveringen.

10 Datablad

Datablad ([12](#)) er vedlagt. Det viser mye statistikk om min mikrokontroller og en del nyttige tips om man skal bruke den.

11 Oppsummering

Som jeg skrev var målet å bruke nios-2 til å styre en selvlaget CPU med data og program, og om mulig også å bruke nios-2 til å vise hva CPUen har gjort for testing samt for data-uthenting.

Selve fabrikeringen av denne enkel mikrokontrolleren var en stor suksess. De utfordringer som oppstod underveis klarte jeg å løse, og alt virker perfekt nå.

VHDL koden ble ganske omfattende så etter at jeg hadde kommentert det meste essensielle, valgte jeg å gå videre og å heller prioritere testing.

Simuleringen ([8](#)) gikk kjempe bra med UVVM ([8.1](#)), hvor jeg måtte endre på noen få ting fordi questa har en annen compiler enn quartus. Samt at UVVM sin SBI virket litt anderledes enn SBI til NIOS-II.

Jeg fikk tid til å simulere 4 programmer. Jeg testet ([9](#)) også de fire samme programmene med NIOS-II og det kunne ikke gått bedre. Men hadde likt å hatt litt mere tid til å gjøre alle målene (prøve å multitaske med flere mikrokontrollere).

Når hvert av programmene virket var det en stor lettelse. Som gjorde at jeg kunne fokusere på dokumenteringen og rapportskriving.

Under programmeringen fant jeg raskt ut at å programmere med binære tall var veldig tungvint samt vanskelig å feilsøke dersom en har gjort skrivefeil. For å komme rundt dette valgte jeg å lage mitt eget kodespråk ([6](#)). Da kunne jeg lett lese hva programmene gjorde, med bonus som økt forståelse.

Så laget jeg med litt hjelp fra Chat-GPT en kompiler ([7](#)) som gjorde at man kan programere textbasert. Programmeringen ble mye lettere etter det.

Selve rapporten er skrevet i Latex som jeg har ønsket å lære meg, så prosjektoppgaven har vært interessant og lærerik på mange måter.

12 Filer

	Fil	Skrevet av	Komentar
Utgangspunkt	Oblig-1	Erlend Sturlason	Jeg klarte ikke å få min Oblig-1 til virke så vi ble enig om at jeg fikk bruke Erlend sin. Den filen blir ble påbygget av Sondre (meg) til å lage prosjektet
Prosjekt-mappe	Prosjekt-ele113-sondre	Sondre Johnsen	Dette er hvor det funksjonerende prosjektet ligger med program-filene
Compiler	compiler_MyMCU_V2.py	Sondre Johnsen	Laget med hjelp fra Chat-GPT [2] 13
	kode.txt	Sondre Johnsen	Denne blir lest av compiler_MyMCU_V2.py
	maskinkode.txt	Sondre Johnsen	Her kommer maskinkoden fra compiler_MyMCU_V2.py ut fra
Indikator	heart.vhd [5] 13	Eivind Skjæveland	En standard fil fra som viser om DE2-115 kortet er programert
Hoved VHDL Kode	adder_8bit.vhd	Sondre Johnsen	
	alu.vhd	Sondre Johnsen	
	DATA_stoom.vhd	Sondre Johnsen	Tok utgangspunkt i count ones reg skrevet av Eivind Skjæveland[3] 13
	decoder.vhd	Sondre Johnsen	
	fulladder.vhd	Sondre Johnsen	
	pointer.vhd	Sondre Johnsen	
	RAM.vhd	Sondre Johnsen	
	ram1x8bit.vhd	Sondre Johnsen	
	ram1x12bits.vhd	Sondre Johnsen	
	ram16x12bit.vhd	Sondre Johnsen	
	ROM.vhd	Sondre Johnsen	
	sCPU_top.vhd	Sondre Johnsen	Dette er topp nivå filen til prosjektet
	selector2bit.vhd	Sondre Johnsen	
	selector4bit.vhd	Sondre Johnsen	
	Mikrokontroller_UVVM	Sondre Johnsen	Dette er Mappen der simuleringen foregikk, Det ligger simuleringsfilen og en kopi av alle .vhd filene
UVVM fil	sCPU_top_UVVM.vhd	Sondre Johnsen	Opprinnelig hentet fra canvas [4] 13 så passet jeg det til mitt bruksområde, den finnes i Mikrokontroller
C filer / mapper:	Foorste-program	Sondre Johnsen	Inni her ligger et C program som tilsvare det Første programmet
	Andre-program	Sondre Johnsen	Inni her ligger et C program som tilsvare det Andre programmet
	Fibonacci-program	Sondre Johnsen	Inni her ligger et C program som tilsvare det Fibonacci programmet
	Count_ones-program	Sondre Johnsen	Inni her ligger et C program som tilsvare det Count-ones programmet
Datablad	Mikrokontroller_Datablad .pdf	Sondre Johnsen	Her ligger databladet

13 Referanse

[1] Lockheedmartin,

lenke: <https://www.lockheedmartin.com/en-us/news/features/2022/orion-redundancies.html>
skrevet 12. juni 2022, hentet 8 mai 2025. fant ikke forfatter.

[2] OpenAI, ChatGPT,

lenke: <https://chat.openai.com/chat>, 6. mai 2025, Spørsmål begynte: "Jeg skal lage en compiler som har en spesiell maskinkode, hvordan leser man fra .txt dokument og sammenligner med en liste instruks selv om noen variabler endrer seg."

[3] Eivind Vågslid Skjæveland, filnavn «COUNT_ONES_REG.vhd» fra Canvas → ELE113-1 25V → Oppgaver: «Oblig-2», (løsningsforslag),

lenke: <https://hvl.instructure.com/courses/28662/assignments/87247>, [Hentet 3. mai]

[4] "usikker på forfatter", filnavn «timer_tb_uvvm.vhd» Canvas → ELE113-1 25V → Sider → «F16 Introduksjon til UVVM 6. mars 2025»,

lenke: <https://hvl.instructure.com/courses/28662/pages/f16-introduksjon-til-uvvm-6-mars-2025>
[Hentet 6. mai]

[5] Svein Haustveit, filnavn «hart.vhd» Canvas → ELE113-1 25V → Sider → «Lab1»,

lenke: <https://hvl.instructure.com/courses/28662/pages/lab1> [Hentet 15. feb]